

CYBERSHIELD

Software Requirements Specification

An Evidence-Driven URL Phishing Detection Platform

Document type	Software Requirements Specification (SRS)
Project status	Implementation completed to date
Prepared for	Academic project documentation
Date	06 August 2026

Acknowledgement

This report records the requirements and current implementation of CyberShield, an academic cybersecurity project. It has been prepared from the implemented source code, user interface, analyzer pipeline, and supporting documentation available at the time of writing. The project acknowledges the guidance of faculty, peers, and all contributors who supported the research, development, testing, and documentation activities.

The document also recognizes the work of the open-source communities behind FastAPI, Playwright, Firebase, Python security libraries, and the web standards on which the prototype is built.

Abstract

CyberShield is a web-based URL phishing analysis platform. It investigates a submitted HTTP or HTTPS URL by collecting live infrastructure and page-behaviour evidence rather than relying only on URL character patterns or brand-name rules. The system validates the target, blocks localhost and private-network destinations, performs HTTP, DNS, domain, TLS, reputation, browser, HTML, JavaScript, and form analysis, then converts the evidence into a stable feature set for an explainable phishing-risk decision.

The current implementation provides a browser dashboard, Firebase-based user accounts and saved scan history, a FastAPI backend, an isolated Playwright browser session, screenshot capture, visible risk indicators, feature breakdowns, and printable analysis reports. The classifier is explicitly documented as an explainable development baseline; a separately generated synthetic-data model may support demonstrations but is not a production blocking model. This SRS defines the implemented scope, functional and non-functional requirements, data requirements, interfaces, constraints, and planned extensions.

Table of Contents

1 Introduction

2 Overall Description

3 System Features and Functional Requirements

4 External Interface Requirements

5 Non-functional Requirements

6 Data Requirements and System Models

7 Validation, Current Status and Future Scope

8 References

Appendix A - Requirement Traceability Matrix

List of Figures

Figure 1. CyberShield high-level architecture

Figure 2. URL analysis data flow

Figure 3. User interaction and result lifecycle

List of Tables

Table 1. Functional requirements

Table 2. External interfaces

Table 3. Quality attributes

Table 4. Persistent data items

Table 5. Requirement traceability matrix

1 INTRODUCTION

- Purpose and scope
- Objectives
- Problem definition
- Document conventions
- Literature and technology context

1.1 Purpose

The purpose of CyberShield is to help a user inspect a potentially unsafe web address before interacting with it. The system presents an understandable phishing-risk result based on evidence observed from the public network, transport security, resolved page, browser behaviour, page markup, scripts, and forms. It is intended as decision support and not as an automatic authority to block or allow access.

1.2 Scope

The scope implemented to date is URL analysis. A user can submit a complete HTTP or HTTPS URL from the dashboard, view the returned analysis, retain it in personal history, revisit it from History or Reports, and print/download the browser result. Email and image tabs are present in the interface but are currently informational stubs and are not included in the implemented analysis scope.

1.3 Objectives

- Collect evidence from several independent sources so a decision is not based only on the URL text.
- Protect the analyzer from server-side request forgery by rejecting localhost, private, link-local, and other non-public destinations.
- Render the target in a controlled browser session and preserve a screenshot tied to that scan.
- Show a risk score, verdict, contributing indicators, and feature values in a user-friendly result page.
- Provide account-based persistence for scan history, reports, settings, and profile information.

1.4 Problem Definition

Phishing websites often imitate trusted services, redirect users across domains, collect credentials, request one-time passwords, use insecure form actions, or conceal risky behaviour in browser-rendered content. Users may see only a link or login page and cannot easily assess its technical properties. CyberShield addresses this gap by making collected risk evidence visible, structured, and reviewable before a user acts on a suspicious URL.

1.5 Definitions and Abbreviations

Term	Meaning
SRS	Software Requirements Specification.
SSRF	Server-Side Request Forgery; an attempt to make the analyzer access internal services.
TLS	Transport Layer Security used to protect HTTPS connections.
DNS	Domain Name System records used to resolve a hostname.
Firestore	Firebase cloud document database used by the frontend for user-owned saved analyses.
Evidence-v1	CyberShield's current stable feature schema derived from collector output.

2 OVERALL DESCRIPTION

- Product perspective
- User classes
- Operating environment
- Constraints and assumptions
- Technology stack

2.1 Product Perspective

CyberShield is a multi-page web application with a separate API service. The frontend provides account and result workflows, while the backend coordinates live collectors. The application is designed as a demonstration-grade security analysis platform: it collects real evidence and clearly labels the current classifier limits.

2.2 High-Level Architecture

The browser sends a submitted URL to the FastAPI analysis endpoint. The orchestrator normalizes the URL, performs a public-target safety check, calls the collectors, derives evidence-v1 features, and requests a risk prediction. The frontend converts the response into a readable result and saves the user-visible record, including the screenshot URL and feature values, to the user's Firestore analysis collection.

Layer	Implemented responsibility	Main technologies
Presentation	Dashboard, history, reports, profile, settings, result rendering and browser print flow.	HTML, CSS, JavaScript, Firebase SDK
API	URL validation, request handling, orchestration, CORS, health endpoint and static screenshot serving.	FastAPI, Pydantic, Uvicorn
Collection	HTTP, DNS, domain, TLS, browser, HTML, JavaScript, form and reputation evidence collection.	httpx, dnspython, WHOIS, Playwright, BeautifulSoup
Decision	Feature extraction, explainable contributors and optional demonstration model loading.	Python, pandas, scikit-learn, joblib
Persistence	Authenticated user profiles, settings and personal analysis records; local API scan records support platform endpoints.	Firebase Authentication, Firestore, SQLite

2.3 User Classes

User class	Characteristics and permissions
Guest	Can access the URL dashboard and use temporary session history. Profile, full History, and Reports are restricted.
Registered user	Uses Firebase Authentication; can save and revisit personal analyses, update profile/settings, and access reports.
Administrator / developer	Operates the local API, monitors diagnostics, configures optional threat-intelligence keys, and maintains the project.

2.4 Operating Environment

- Frontend: current desktop or mobile browser, with Chrome, Edge, or Firefox expected to support modern JavaScript and CSS.
- Backend: Python 3 environment capable of running FastAPI and Playwright Chromium. Windows is supported through a Proactor event loop for browser subprocess compatibility.
- Network: internet access is required for public DNS, HTTP/TLS, WHOIS, optional reputation providers, and Firebase cloud services.

2.5 Constraints and Assumptions

The analyzer only accepts complete public HTTP/HTTPS URLs. Live analysis quality depends on external websites, DNS resolution, TLS availability, browser compatibility, optional third-party keys, and network connectivity. A scan may return partial browser evidence when rendering fails; that failure must be shown as unavailable evidence rather than fabricated data. The user must understand that the current risk probability is advisory.

3 SYSTEM FEATURES AND FUNCTIONAL REQUIREMENTS

- Authentication and account workflow
- URL validation and analysis
- Risk decision and result presentation
- History and reports
- Administration and diagnostics

3.1 Functional Requirements

ID	Feature	Requirement
FR-01	Account access	The system shall allow registration, sign-in, password-reset flow, sign-out, and guest access through Firebase Authentication.
FR-02	URL admission	The system shall accept only complete HTTP/HTTPS URLs and reject malformed, localhost, private, link-local, reserved, or otherwise non-public targets before collection.
FR-03	Infrastructure analysis	The system shall collect HTTP reachability/redirect data, DNS data, domain age/registration data, and TLS certificate metadata when available.
FR-04	Browser analysis	The system shall open an admitted URL in a controlled Playwright browser context and collect the final URL, title, DOM, browser events, and a scan-specific screenshot when possible.
FR-05	Content analysis	The system shall inspect rendered HTML, JavaScript, and forms for observable phishing-relevant signals, including hidden iframes, obfuscated scripts, login form behaviour, OTP fields, card fields, insecure actions, and cross-domain form actions.
FR-06	Risk result	The system shall produce an explainable probability, risk class, verdict, and a ranked set of evidence contributors.
FR-07	Result display	The system shall display the risk score, verdict, analyzed input, timestamp, recommendation, indicators, feature breakdown, and screenshot when a screenshot exists for that scan.
FR-08	Persistence	The system shall save registered-user analyses with input type, content, score, verdict, indicators, features, screenshot URL, and timestamps in the user's Firestore collection.
FR-09	History and reports	The system shall list saved analyses and shall load the complete saved result, including features and screenshot URL, when a user selects View or View full result.
FR-10	Reporting	The system shall permit a saved result to be printed or saved as a PDF using the browser print facility.
FR-11	Settings and profile	The system shall allow an authenticated user to view/update supported profile fields and save report preferences.
FR-12	Service health	The API shall expose root and health endpoints that identify service status.

3.2 URL Analysis Workflow

- Normalize the submitted address and validate the URL syntax.
- Resolve the hostname and reject any target that is local or non-public.
- Collect HTTP, DNS, domain, TLS, and optional reputation evidence.
- Render the page in the secure browser context, capture a screenshot, and inspect page content, scripts, and forms.
- Extract evidence-v1 features, calculate the phishing probability, identify contributors, and return the result.
- Persist the user-facing result and route the user to the complete analysis page.

3.3 Risk Classification Rules

The current baseline starts from a conservative base score and adds weighted evidence only when a collector supplies a value. Example evidence includes recent domain or certificate age, redirect chains, risky credential forms, OTP/card fields, hidden iframes, meta refresh redirects, downloads, permission prompts, obfuscated scripts, and configured reputation detections. The probability is mapped to Safe, Low, Medium, High, or Critical risk ranges. The frontend uses Safe below 40, Suspicious from 40 to 69, and Phishing at 70 or above for its user-facing verdict banner.

4 EXTERNAL INTERFACE REQUIREMENTS

- User interface
- Software interfaces
- API interfaces
- Communication interfaces

4.1 User Interface Requirements

Screen	Required behaviour
Landing and authentication	Provide a clear entry point, sign-in, registration, password recovery, and guest access.
Dashboard	Provide a URL input, analysis progress states, current counters, recent history, and navigation to account features.
Analysis Result	Show a readable verdict banner, risk score, input metadata, recommendation, indicators, feature table, scan-specific screenshot, and report actions.
History	Provide filtering/searching, scan metadata, View full result, and deletion of the user's saved analysis.
Reports	Show saved report rows and provide View access to the same complete result data as a newly completed scan.
Profile and Settings	Allow account-related information and preferences to be viewed or changed according to authenticated access.

4.2 API Interfaces

Endpoint	Method	Purpose
/analyze	POST	Runs synchronous URL analysis and returns the standardized analysis response.
/api/auth/register	POST	Creates a local platform user and signed token for the platform API workflow.
/api/auth/login	POST	Authenticates a local platform user and returns a signed bearer token.
/api/scans	POST / GET	Submits an asynchronous scan or retrieves scan history in the platform API workflow.
/api/scans/{scan_id}	GET	Returns the status/result of a queued scan after authorization checks.
/health	GET	Returns healthy/online service status.
/reports	Static mount	Serves generated analysis screenshot files for result rendering.

4.3 Communication Interfaces

The browser frontend communicates with the local FastAPI backend using HTTP requests and JSON. The backend communicates with public internet services over HTTP/HTTPS, DNS, TLS sockets, and WHOIS-related lookup mechanisms. Firebase Authentication and Firestore are accessed from the frontend through Firebase web modules. The deployment must configure CORS origins to allow only known frontend origins.

4.4 Software Interfaces

- FastAPI and Pydantic provide API routing and typed request/response models.
- Playwright Chromium renders target pages in a controlled browser context.
- BeautifulSoup and lxml support HTML parsing; httpx supports outbound HTTP.
- Firebase Authentication and Firestore support the production-facing frontend account and history experience.
- SQLite supports the implemented backend platform scan/account endpoints for local workflows.

5 NON-FUNCTIONAL REQUIREMENTS

- Security
- Performance and reliability
- Usability
- Maintainability
- Compliance and ethical use

5.1 Quality Attributes

ID	Attribute	Requirement
NFR-01	Security	Reject unsafe internal/private targets before HTTP, TLS, or browser collection to reduce SSRF exposure.
NFR-02	Security	Use Firebase Authentication for frontend session identity; enforce Firestore rules so users access only their records.
NFR-03	Privacy	Store only information needed for the scan record. A captured screenshot must be associated with the specific analysis that generated it.
NFR-04	Reliability	Return partial, labelled evidence when a collector is unavailable; do not convert missing evidence into a positive or negative claim.
NFR-05	Performance	Provide visible progress during analysis and avoid blocking the web server's primary event loop while Playwright creates browser subprocesses.
NFR-06	Usability	Present a concise verdict, explanation, risk score, and recommendation in language understandable to a non-specialist.
NFR-07	Maintainability	Keep collectors, feature extraction, classification, and frontend rendering separated so sources and weights can evolve independently.
NFR-08	Auditability	Preserve the input, timestamp, verdict, indicators, features, and screenshot reference for authenticated users' saved scans.
NFR-09	Portability	Support local development through a Python API and static frontend server, with a Docker Compose option for containerized setup.
NFR-10	Ethical use	Describe the current classifier as an advisory baseline. Do not represent a synthetic-data demonstration model as a validated production defence.

5.2 Security Requirements

A security analyzer is itself exposed to hostile input. CyberShield shall treat every submitted URL as untrusted, block unsafe address classes before network access, use request timeouts, isolate webpage rendering from the user interface, and avoid exposing secret API keys in source control. Optional reputation providers must remain disabled until their keys are configured. User data access in Firestore must be scoped to the authenticated UID.

5.3 Reliability and Error Handling

If a target cannot be reached, a third-party lookup is unavailable, or browser analysis fails, the result shall preserve the error state and available evidence. The backend shall close browser resources in a finally path. The frontend shall hide a screenshot area when the requested image fails to load rather than displaying a wrong shared image.

6 DATA REQUIREMENTS AND SYSTEM MODELS

- Analysis response
- Feature schema
- Persistent data
- Data flow

6.1 Analysis Response Data

The primary response contains success state, original and normalized URL, reachability, phishing probability, risk, message, and a data object. The data object holds source-specific evidence, the feature payload, the classifier output, and browser capture metadata.

Data group	Representative elements	Purpose
Safety	allowed, resolved_addresses	Documents URL admission checks and resolved public addresses.
Infrastructure	HTTP, DNS, domain, SSL/TLS, reputation	Provides observable network and registration evidence.
Browser	final URL, title, redirects, events, screenshot, browser error	Records browser-rendered page behaviour and capture state.
Content	HTML, JavaScript, form analysis	Supplies page and credential-collection indicators.
Features	schema_version, values, available_sources	Produces stable model-ready evidence-v1 values.
Classification	model version/status, probability, risk, contributors, notice	Provides explainable user-facing decision information.

6.2 Evidence-v1 Feature Families

- Domain and TLS: domain age, certificate age, and derived recency risks.
- Network: DNSSEC availability, HTTP reachability, redirect count, and TLS availability.
- Forms: password, OTP, and payment-card fields; cross-domain and insecure form actions; GET login forms.
- Page and browser: hidden iframes, meta refresh, popups, downloads, permission requests, and JavaScript errors.
- Threat intelligence: detection count and Safe Browsing flag when providers are configured.

6.3 Persistent User Analysis Record

Field	Description	Retention/use
inputType	Type of user input; URL is currently supported.	Used to render appropriate result metadata.
content	Submitted URL.	Shown in history and result metadata.
riskScore and verdict	User-facing assessment outputs.	Used for lists, statistics, filters, and result banner.
indicators and features	Contributors and stable feature values.	Used to recreate the complete result detail.
screenshot Url	URL for the capture generated by this scan.	Used only for the matching result view.
analyzedAt and createdAt	Scan and persistence timestamps.	Used for ordering, display, and reporting.

6.4 Data Flow

User -> Dashboard -> POST /analyze -> URL safety gate -> evidence collectors -> feature extractor -> explainable classifier -> normalized frontend result -> Firestore user analysis record -> Result / History / Reports. A selected saved record is reconstructed with the same indicators, features, and screenshot URL before the Result page renders it.

7 VALIDATION, CURRENT STATUS AND FUTURE SCOPE

- Validation approach
- Implemented status
- Known limitations
- Future enhancements

7.1 Requirement Validation Approach

Validation is performed through source-level checks, API execution, browser workflow inspection, and result-data consistency checks. The project includes decision-pipeline tests for safety and classification behaviour. Manual acceptance checks confirm registration/guest paths, a successful public URL scan, result rendering, screenshot display, history filtering, report opening, and deletion behaviour.

7.2 Implemented Status

Area	Status	Notes
Public URL safety gate	Implemented	Blocks localhost and non-public IP destinations before collectors run.
Live infrastructure collection	Implemented	HTTP, DNS, domain, TLS, and optional reputation evidence are collected where available.
Controlled browser collection	Implemented	Playwright captures rendered evidence and a scan-specific screenshot.
Explainable risk output	Implemented	Evidence baseline is active; an optional synthetic-data demo model is not production validated.
Frontend account/history/report workflow	Implemented	Firebase account and Firestore persistence are used by the frontend.
Complete saved result reconstruction	Implemented	History and Reports pass indicators, features, and the screenshot URL to the Result page.
Email and image analysis	Planned	Interface tabs are present but currently show an availability notice.
OpenPhish and URLhaus feeds	Planned	Provider placeholders are present; integration is pending.

7.3 Known Limitations

- A risk score is not a guarantee of safety or maliciousness; human verification remains necessary.
- The baseline classifier must be trained and evaluated on labelled benign and phishing samples before automated blocking is

considered.

- Website rendering and external lookups can fail because of target-side restrictions, network conditions, or provider configuration.
- Screenshot availability is dependent on successful browser capture and continued availability of the served capture file.
- The frontend currently uses Firebase identity/history while the backend also contains a separate local platform account/scan API path; deployment should select and consolidate the intended production identity strategy.

7.4 Future Scope

- Integrate and poll VirusTotal, Google Safe Browsing, OpenPhish, and URLhaus according to their service policies.
- Collect consented, labelled evidence rows; train versioned models; measure precision, recall, false-positive rate, and calibration.
- Add email/text and uploaded-image analysis with equivalent persistence and explanation quality.
- Introduce background jobs, queue monitoring, retry policy, retention controls, and production observability.
- Harden deployment with secret management, HTTPS, restrictive CORS, Firebase Security Rules review, rate limiting, and security logging.

8 References

- I. Sommerville, Software Engineering, Pearson. Guidance for requirements engineering and SRS structure.
- ISO/IEC/IEEE 29148, Systems and software engineering - Life cycle processes - Requirements engineering.
- FastAPI Documentation, <https://fastapi.tiangolo.com/>.
- Playwright Python Documentation, <https://playwright.dev/python/>.
- Firebase Documentation, <https://firebase.google.com/docs/>.
- OWASP Server-Side Request Forgery Prevention Cheat Sheet, <https://cheatsheetseries.owasp.org/>.
- Google Safe Browsing API Documentation, <https://developers.google.com/safe-browsing/>.
- CyberShield project source code and README, current workspace implementation, accessed on the report date.

Appendix A - Requirement Traceability Matrix

Requirement	Implementation evidence	Validation method
FR-02	url_safety.py and orchestrator admission path	Submit malformed, localhost, and private-address URLs; verify denial before collection.
FR-03 to FR-05	Analyzer modules and orchestrator collector sequence	Run an admitted public URL and inspect returned source evidence.
FR-06	FeatureExtractor and EvidenceClassifier	Verify feature values, probability, risk, and contributor list in the response.
FR-07	result.html and result.js	Confirm score, indicators, features, recommendation, metadata, and matching screenshot render.
FR-08 to FR-09	firebase-data.js, history.js, reports.js	Save a result, reopen it from History and Reports, and compare full detail with fresh result.
NFR-04	Browser exception handling and available_sources	Simulate/observe a browser failure and verify partial evidence is labelled.
NFR-10	README and classifier notice	Review that baseline/demo model limits are presented to users/developers.

Appendix B - Formatting Compliance

This report follows the visual conventions observed in the supplied Expirochain report: A4 portrait pages, one-inch margins, Arial typography, large readable body text, bold numbered chapter and section headings, a running project header, centered page numbering, consistent table padding, and controlled white space. The SRS content itself follows a requirements-oriented structure so that implemented functionality, current boundaries, and future work remain distinguishable.